```
import Mathlib

set_option maxHeartbeats 2000000
open Polynomial
/--
Let ℚ be the set of rational numbers. A function f : ℚ → ℚ is called aquaesulian if the following property holds: for every x, y ∈ ℚ,

f(x + f(y)) = f(x) + y      or      f(f(x) + y) = x + f(y).

Show that there exists an integer c such that for any aquaesulian function f there are at most c different rational numbers of the form f(r)+f(−r) for some rational number r.
-/
theorem imo_2024_p6
    (IsAquaesulian : (ℚ → ℚ) → Prop)
    (IsAquaesulian_def : ∀ f, IsAquaesulian f ↔ ∀ x y, f (x + f y) = f x + y ∨ f (f x + y) = x + f y) :
    IsLeast {c : ℤ | ∀ f, IsAquaesulian f → {f r + f (-r) | (r : ℚ)}.Finite ∧ {f r + f (-r) | (r : ℚ)}.ncard ≤ c} 2 := by
  exists@?_
  · useλu b=>if j:u 0=0then by_contra λc=>?_ else ?_
    · suffices:({J|∃k,u k+u (-k)= J}) ⊆{0}
      · simp_all[this.antisymm]
      rintro - ⟨a, rfl⟩
      contrapose! c
      simp_all
      suffices:{U|∃examples6, (u) ‹ℚ› +u ( -‹_›)= U} ⊆{0,(u (a : Rat)+ (u<|@@↑(( (-a ))))) } ..
      · use ( Set.toFinite ( _) ).subset ↑@@this , (Set.ncard_le_ncard$ (((this )) ) ).trans (Set.ncard_pair$ Ne.symm (↑
        ↪ ( (c)) ) ).le
      rintro-⟨hz, rfl⟩
      induction b @hz a
      · have:=b (-a)$ hz+u a
        have:=b hz hz
        simp_all[add_comm]
        have:=b (-hz) (hz+u ↑(hz))
        simp_all[ add_assoc, C]
        induction this
        · simp_all
          have:=b hz (hz+(u a+u (-a)))
          have:=b (hz+(u a+u (-a)))$ hz+(u a+u (-a))
          use .inr$ by_contra$ by hint
        have:=b hz$ hz+(u hz+u (-hz))
        cases b (hz+(u hz+u (-hz)))$ hz+(u hz+u (-hz))with|_=>hint
      have:=b (-hz) (u hz+a)
      have:=b$ -a
      specialize this (u hz+a)
      simp_all[ ←add_assoc]
      have:=b 0
      have:=b
      specialize b a a
      simp_all[add_comm]
      have:=(this<| -a) (↑a + (((u a))): (↑_ :((( _) ) ) )) ..
      simp_all[add_assoc]
      cases this
      · simp_all
        contrapose! IsAquaesulian_def
        simp_all
        exfalso
        have:=this a (a+(u hz+u ( -hz)))
        simp_all[Ne.symm,Bool]
        have:=‹∀congr_arg G,_› (a+(u hz+u (-hz)))$ a+(u ↑hz+u ↑( -hz) )
        simp_all
      have:=this a (a +(u a+u (-a)))
      cases‹forall Jd S,_› (a+(u a+u (-a))) ( a + (u a +u ↑(-a)))with| _ =>hint
    simp_all
    cases b 0 0with|_=>exact absurd (b 0$ (0+(1 *(@(u ↑.((0) )))))^ 01: ↑ ((_)) ) (id$ (by(cases ( b (u 0) ( (u
      ↪ 0)))with|_ => continuity)))
  rintro K V
  specialize V $ λ N=>-N+2 *Int.ceil N
  specialize( V $ (IsAquaesulian_def _).mpr _)
  · simp_rw [ ←eq_sub_iff_add_eq']
    ring
    use mod_cast@?_
    norm_num[<-add_mul,Int.ceil_eq_iff]
    useλc K=>(em _).imp (⟨by linarith[Int.ceil_lt_add_one c,Int.le_ceil K],.⟩) (by repeat use by linarith[.,Int.le_ceil
      ↪ c,or,Int.ceil_lt_add_one$ K])
  simp_all[Int.ceil_neg, ←add_assoc]
  suffices:2<=V.1.toFinset.card
  · let M:=V.1.toFinset
    norm_num[this,V.2.trans',(Set.ext$ by simp_all[M]: {x :Rat|∃t:Rat, (↑2 ) * ( ⌜ t ⌝:(ℚ ) ) .. + (- (2 *L(t)⌋)) = ↑x} =
      ↪ M)]
  use Finset.one_lt_card.2$ by exists@0,V.1.mem_toFinset.2 (by exists-1),2,V.1.mem_toFinset.2 (by exists 1/2)
```